A. D. PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

Quillkey: Adaptive Typing Speed Tester

A Python Mini Project Report

A report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology
In
Information Technology
Semester – V

Subject:
Programming with Python (202044504)

By
Mansi Baraiya (12402080601075)

Faculty Guide
Nayan M. Mali
Assistant Professor

Academic Year 2026–27

Department of Information Technology
A D Patel Institute of Technology
New Vallabh Vidyanagar, Vitthal Udyognagar – 388121
Anand (Gujarat), India.

CERTIFICATE

This is to certify that the mini project work embodied in this report entitled “Quillkey: Adaptive Typing Speed Tester” was carried out by Mansi Baraiya (12402080601075) at A. D. Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree awarded by the Charutar Vidya Mandal (CVM) University. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: _____
Place: New Vallabh Vidyanagar

Lab Teacher
Nayan M. Mali
Assistant Professor

Head of Department
Dr. Priyang Bhatt
Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the opportunity and resources to undertake this mini project, “Quillkey: Adaptive Typing Speed Tester”.

I am thankful to my faculty guide and lab teacher for their guidance, suggestions and support throughout the development of this project. Their feedback helped me understand the design, implementation and testing of the system.

I also thank my classmates and everyone who provided assistance or encouragement during the project. Finally, I am grateful to my family for their continuous support.

Mansi Baraiya
(12402080601075)

TABLE OF CONTENTS

CERTIFICATE **2**
ACKNOWLEDGEMENT **3**
TABLE OF CONTENTS AND LIST OF TABLES **4**
ABSTRACT **5**
CHAPTER 1: INTRODUCTION **6**
1.1 Background 6
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Scope and Applications 6
CHAPTER 2: THEORETICAL BACKGROUND **6**
2.1 Strings 6
2.2 Functions 7
2.3 Graphical User Interface (GUI) 7
2.4 Supporting Concepts: HTTP, JSON and SQLite 7
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY **7**
3.1 System Overview 7
3.2 Working of the System 7
3.3 Core Logic and Formulas 8
3.4 Database and API Design 8
CHAPTER 4: IMPLEMENTATION AND TESTING **8**
4.1 Technologies Used 8
4.2 Implementation 8
4.3 User Interface 8
4.4 Testing 9
4.5 Result and Discussion 9
CHAPTER 5: FUTURE SCOPE AND CONCLUSION **10**
CHAPTER 6: REFERENCES **10**

LIST OF TABLES

Table 1: Files and responsibilities of the project
Table 2: REST API endpoints
Table 3: Technologies used in the project
Table 4: Test cases and results

ABSTRACT

Quillkey is an adaptive typing speed tester built to demonstrate three core Python topics: strings, functions and graphical user interface (GUI) design. A learner types a short practice text while the application measures speed in words per minute (WPM), accuracy, the number of errors and elapsed time, and updates these values live on screen. After every session the result is saved, and the difficulty of the next text is adjusted automatically.

The project follows a client–server design. A Python backend, written with only the standard library (http.server and sqlite3), generates the practice text from a set of unlocked letters, stores sessions and per-letter statistics in an SQLite database, and decides whether the learner moves up or down a level. A browser-based GUI built with HTML, CSS and JavaScript provides the typing screen, a results screen and a progress dashboard with a speed chart and a letter-accuracy map. Letters the learner misses most often are shown more frequently, so practice focuses on weak keys. The project needs no external packages and runs locally with a single command.

CHAPTER 1: INTRODUCTION

1.1 Background

Typing is a basic skill for programmers, students and office workers. Speed and accuracy improve mainly through regular, focused practice, but ordinary typing tests give the same kind of text to everyone and do not react to the learner's mistakes. A beginner can feel overwhelmed by a full keyboard, while an experienced typist can get bored with easy text.

Quillkey takes a different approach. Practice starts with only six common letters (e, n, i, t, r, l) and unlocks one new letter at a time as the learner proves speed and accuracy. The practice text is made of pseudo-words built from the unlocked letters, which is a good use of Python string handling, while the program logic is organised into small functions and presented through a GUI.

1.2 Problem Statement

Learners need a typing practice tool that measures their performance in real time, remembers their progress between sessions, and adapts the difficulty to their ability. The problem addressed in this project is to design and build such a tool in Python, using string operations to generate and process text, functions to structure the logic, and a graphical interface to show the typing area, statistics and progress.

1.3 Objectives

- To generate practice text dynamically from a growing set of letters using Python string operations.
- To measure WPM, accuracy, errors and time live while the user types.
- To structure the program into reusable functions for text generation, saving results, level control and progress reports.
- To build a GUI with a typing screen, a results screen and a progress dashboard with a chart.
- To store sessions and per-letter statistics permanently in an SQLite database.
- To adapt difficulty: unlock a letter after a fast and clean session, and step back after a poor one.

1.4 Scope and Applications

The scope is a single-user application running on the local machine at http://127.0.0.1:8000. It practises lowercase letters only (no digits or punctuation), uses no external libraries and needs no internet connection. It can be used by students learning touch typing, as a warm-up tool for programmers, and as a teaching example that shows how Python can serve a web interface and a database without any framework.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Strings

A string in Python is an immutable sequence of characters. Strings support indexing and slicing, concatenation, repetition and a rich set of methods. Because every string is a sequence, a program can walk through it character by character, which is exactly what a typing tester does when it compares each key pressed with the expected character. The project uses these string features:

- **Slicing:** ORDER[:START + lv] takes the first few letters of the unlock-order string "enitrlsauodychgmpbkvwfzxqj".
- **Concatenation and join:** a word is built letter by letter with word += letter, and the words are joined into one sentence with " ".join(words).
- **Methods and membership:** isalpha(), len() and the in operator validate letter keys, while lower() makes key statistics case-insensitive.
- **Formatting:** f-strings in Python and template literals in JavaScript produce messages such as the "letter unlocked" note.

2.2 Functions

A function is a named block of code that takes input, performs one task and returns a result. Functions remove repetition, make testing easier and keep each part of a program small and readable. Python also allows a function to be defined inside another one, and the project uses this for the small helper num() inside save(), which limits a value to an allowed range. Default values, return statements, exception handling with try/except and lambda-style arrow functions in JavaScript are all used. In this project each responsibility has its own function, as listed in Section 4.2.

2.3 Graphical User Interface (GUI)

A GUI lets the user work with visual elements such as buttons, text areas and charts instead of typing commands. A GUI is event-driven: the program waits for events such as a key press or a click and runs a handler function for each. Python offers desktop toolkits such as Tkinter, but this project builds a browser-based GUI. The interface is written in HTML (structure), CSS (appearance, light and dark themes, responsive layout) and JavaScript (event handling and drawing), and it talks to the Python backend through fetch() calls. This keeps the Python code focused on data and logic and makes the interface work on desktop and mobile browsers.

2.4 Supporting Concepts: HTTP, JSON and SQLite

The browser and the Python server communicate over HTTP. The server sends and receives data in JSON, a text format that maps directly to Python dictionaries and lists through the json module. SQLite is a file-based database included with Python (module sqlite3), so progress is saved without installing a database server. Queries use ? placeholders so that user input can never be run as SQL code.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 System Overview

The system has three layers: the GUI in the browser, the Python HTTP server with its API, and the SQLite database. Table 1 lists the files and what each one does.

File	Responsibility
server.py	HTTP server, JSON API, practice-text generator, level logic and database access.
index.html	Structure of the three views: practice, results and progress.
style.css	Styling, responsive layout and automatic light/dark theme.
app.js	Typing engine, live statistics, results screen, dashboard and canvas chart.
quillkey.db	SQLite file created automatically; holds sessions, key statistics and the current level.

Table 1: Files and responsibilities of the project

3.2 Working of the System

Workflow: Start server → Open page → Request text (GET /api/text) → Type and update live stats → Finish text → Send result (POST /api/session) → Server saves and updates level → Show results → View progress (GET /api/progress) or practise again.

When the page opens, app.js requests a text of about 140 characters. The server reads the current level, builds the text and returns it with the unlocked letters. The text is displayed one character per element, and the first character is highlighted. The timer starts on the first key press. Every key is compared with the expected character: a correct key turns the character green and moves the cursor on, while a wrong key counts as an error and flashes red, and the cursor stays until the correct key is typed. When the last character is typed, the result and per-letter hit/miss counts are posted to the server, which replies with the new level, whether the learner set a personal best, and the earlier average.

3.3 Core Logic and Formulas

- **Speed:** WPM = (correct characters ÷ 5) ÷ minutes elapsed. Five characters are counted as one standard word.
- **Accuracy:** Accuracy % = correct ÷ (correct + errors) × 100.
- **Level rules:** accuracy ≥ 95% and WPM ≥ 20 → level + 1 (a new letter is unlocked); accuracy < 85% → level − 1; otherwise the level is unchanged. The level is limited to 0–20, so the letters grow from 6 to all 26.
- **Weak-letter weighting:** each unlocked letter gets weight = 1 + 8 × (miss rate) + 2 (if it is the newest letter). Letters are then picked with random.choices(), so frequently missed and newly unlocked letters appear more often.
- **Word shape:** words are 3 to min(8, 5 + level ÷ 4) letters long and mostly alternate vowels and consonants (switching with probability 0.75), which keeps the pseudo-words easy to pronounce and type.

3.4 Database and API Design

The database has three tables: sessions (timestamp, WPM, accuracy, errors, seconds, characters, level), keys (letter, hits, misses) and state (key–value pairs, used to store the current level). The server exposes the endpoints in Table 2.

Endpoint	Method	Purpose
/api/text?chars=N	GET	Returns generated text, current level and unlocked letters. N is limited to 40–400 (default 140).
/api/session	POST	Saves a finished session and per-letter counts; returns new level, personal-best flag and average.
/api/progress	GET	Returns the last 100 sessions, letter statistics, level and total practice time.

Table 2: REST API endpoints

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Technologies Used

Technology	Purpose
Python 3.8+ (http.server)	Web server, API and application logic using only the standard library.
sqlite3, json, random, time	Database storage, data exchange, random text generation and timestamps.
HTML5, CSS3	Structure and styling of the GUI, including dark mode and responsive layout.
JavaScript, Canvas API	Typing engine, event handling, live statistics and the speed chart.

Table 3: Technologies used in the project

4.2 Implementation

Python functions (server.py): db() opens a connection; init() creates the tables; level() reads the saved level; make_text(n) builds the weighted pseudo-word text; save(d) validates and stores a session and applies the level rules; progress() prepares the dashboard data. The Handler class serves the static files and routes /api/ requests in do_GET and do_POST; requests for .py and .db files are refused with 404 so the source and database cannot be downloaded.

JavaScript functions (app.js): newGame() fetches text and resets the state; press(ch) checks each key and updates counts; stats() recalculates WPM, accuracy and time every 200 ms; finish() posts the result and fills the results screen; loadDash() and drawChart() build the summary cards, letter-accuracy map and the line chart. A hidden textarea receives all typing, so the same code works with mobile keyboards.

4.3 User Interface

The application has three views reached from the top navigation. The Practice view (Figure 1) shows the current level, the unlocked letters with the newest one highlighted, four live statistic cards (WPM, accuracy, errors, time)

and the text with the current character highlighted. Pressing Esc loads new text. The Results view shows WPM, accuracy, errors, time and characters, a "New personal best!" message when applicable, and a note about unlocking or dropping a letter. The Progress view shows summary cards, a speed-over-time chart and a per-letter accuracy map in which locked letters are greyed out.



Figure 1: Practice screen at Level 4 (letters e n i t r l s a u unlocked, new letter "u" highlighted)

4.4 Testing

The backend functions and API were tested with a temporary database so that the real progress file was not changed. The results are shown in Table 4. All tests passed.

No.	Input / Action	Expected Result	Observed Result
1	make_text(140) at level 0	Only letters e n i t r l are used	PASS: letters "enitrl", 140 characters
2	Session: 25 WPM, 97% accuracy	Level 0 → 1, letter "s" unlocked	PASS: level 1, change "up", letter "s"
3	Session: 15 WPM, 97% accuracy	Level unchanged (too slow)	PASS: level stays 1
4	Session: 30 WPM, 80% accuracy	Level drops by one	PASS: level 1 → 0
5	Out-of-range values, keys "1" and "ab"	Values clamped, non-letters ignored	PASS: WPM 400, accuracy 100, only e and n stored
6	Letter "e" missed 90% of the time	"e" appears more than other letters	PASS: e = 136 of 325 letters; others 18–48
7	25 clean sessions in a row	Level stops at the maximum (20)	PASS: level 20, 26 letters
8	GET /api/text?chars=5	Minimum length applied	PASS: status 200, text of 39 characters
9	GET /server.py, /quillkey.db	Access blocked	PASS: both return 404
10	POST /api/session with invalid JSON	Error, no crash	PASS: status 400

Table 4: Test cases and results

4.5 Result and Discussion

The tests show that the three project topics work together correctly. String slicing and joining produce text that contains only the unlocked letters; the functions apply the level rules consistently and safely; and the GUI presents live feedback and saved progress. In the sample screenshot the learner is at Level 4 with nine letters unlocked, which matches the saved level (3) in the database, because the interface shows level + 1. The weighting test confirmed that a letter with a high miss rate is repeated about three times as often as the other letters. One small observation is that a requested minimum of 40 characters gives a text of 39 characters, because the final space is not counted; this does not affect practice.

CHAPTER 5: FUTURE SCOPE AND CONCLUSION

5.1 Future Scope

The project can be extended with capital letters, digits and punctuation levels; real words and sentences instead of pseudo-words; user accounts so that several learners can share one server; a countdown or timed test mode; sound feedback; and export of progress to a file. A desktop version could also be made with Tkinter using the same functions.

5.2 Conclusion

Quillkey shows how strings, functions and a GUI can be combined into a useful application. String operations generate adaptive practice text, small single-purpose functions handle text creation, saving, levelling and reporting, and a browser GUI gives live feedback and a progress dashboard. The program uses only the Python standard library, stores progress in SQLite and passed all of its tests. It is a practical learning project and a base for a more complete typing trainer.

CHAPTER 6: REFERENCES

1. Python Software Foundation. The Python Standard Library: http.server, sqlite3, json and random modules, docs.python.org.
2. Python Software Foundation. The Python Tutorial: Strings and Defining Functions, docs.python.org.
3. SQLite Documentation, sqlite.org/docs.html.
4. MDN Web Docs. HTML, CSS, JavaScript, Fetch API and Canvas API reference, developer.mozilla.org.
5. Lutz, M. Learning Python, 5th Edition, O'Reilly Media, 2013.